

ROLE

You are a Senior Data Architect specializing in Text-to-SQL RAG systems. Your task is to convert a database schema into an optimized RAG context document.

INPUT

I am providing you with my database schema in the form of: [PNG IMAGE / draw.io XML / SQL DDL / CSV headers].

Additional context about my database:

- Database Engine: [MySQL / PostgreSQL / SQLite / SQL Server / BigQuery]
- Database Name: [your_db_name]
- Business Domain: [e-commerce / healthcare / finance / logistics / SaaS]
- Data Time Range: [e.g., 2017-2024 / real-time / historical only]
- Approximate Size: [e.g., 100K rows in main table]

TASK

Generate a complete RAG context document in Markdown format. This document will be injected into an LLM prompt to convert natural language questions into SQL queries. It must be token-efficient, precise, and unambiguous.

REQUIRED OUTPUT STRUCTURE

Produce the document with exactly these 8 sections:

Section 1: Database Schema

For EVERY table found in the schema:

- A Markdown table listing: Column Name | Data Type | Key (PK/FK/UNIQUE) | Description
- The equivalent CREATE TABLE DDL statement in a code block
- A one-line plain-English description of what the table stores
- Flag any column whose name is misleading, misspelled, or non-obvious

Section 2: Table Relationships

- An ASCII/text ERD showing table connections with cardinality (1:1, 1:N, N:M)
- A Markdown table of all join paths: From Table | To Table | Join Condition
- The canonical full-join SQL pattern connecting all tables
- Explicitly warn about any column that looks joinable but is NOT a valid foreign key

Section 3: Business Term Definitions

A Markdown table mapping vague business language to exact SQL: | Term | Definition | SQL Formula |
Infer domain-appropriate metrics from the schema. Include at minimum:

- Revenue / total value calculations
- Count metrics (customers, orders, transactions)
- Time-window filters (last week, last month, YTD) using the correct engine syntax
- Average / ratio metrics
- Status-based filters based on actual enum-like columns present

Section 4: Example Queries (Few-Shot)

Provide 6-8 examples in this format:

Q: "[natural language question]"

[correct SQL query]

Cover this range of complexity:

- Simple COUNT on a single table
- Filtered aggregation
- Two-table JOIN
- Full multi-table JOIN with GROUP BY
- Time-window analysis
- Percentage / ratio calculation
- Ranked TOP-N result

Section 5: Security & Generation Rules

- Numbered list of hard constraints (SELECT-only, no DDL/DML, mandatory LIMIT, no comments, no stacked statements, explicit JOIN syntax, table aliasing convention)
- An Ambiguity Handling table: | Ambiguous Term | Clarification Question | Options | Identify every term in this domain that has multiple valid interpretations

Section 6: Common Query Patterns

Reusable SQL snippets grouped by category:

- Time filters (using the correct syntax for the specified database engine)
- Aggregation patterns
- Status/category filters
- NULL handling patterns
- Deduplication patterns

Section 7: Database Statistics

Table of: Table Name | Approximate Rows | Notes Include a warning if the data is historical rather than live.

Section 8: Common Mistakes to Avoid

A two-column table: | Mistake | Correction | Focus on traps specific to THIS schema, such as:

- Columns that look like the right join key but are not
- Metrics that require summing multiple columns
- Tables that must be joined through an intermediate table
- Duplicate-causing joins
- Misspelled column names in the source schema

CRITICAL CONSTRAINTS

1. ACCURACY: Transcribe column names EXACTLY as they appear, including typos, abbreviations, and non-English words. Never "fix" or normalize a name. If the schema says `product_name_lenght`, write `product_name_lenght`.
2. DIALECT: Use SQL syntax valid for the database engine I specified. Do not mix dialects.
 - MySQL: `DATE_SUB(NOW(), INTERVAL 7 DAY)`
 - PostgreSQL: `NOW() - INTERVAL '7 days'`
 - SQLite: `datetime('now', '-7 days')`
 - SQL Server: `DATEADD(day, -7, GETDATE())`
3. NO HALLUCINATION: Only document tables and columns that actually exist in my input. If a data type is unreadable or missing, write `UNKNOWN` rather than guessing.
4. TOKEN EFFICIENCY: Be dense and precise. No filler prose, no marketing language, no restating the obvious.
5. AMBIGUITY FIRST: The single most valuable part of this document is Section 3 and the Ambiguity Handling table in Section 5. Invest the most effort there.
6. UNCERTAINTY FLAGGING: If you cannot confidently read something from the input, add a section at the very end titled " Items Requiring Human Verification" and list them as bullet points.

OUTPUT FORMAT

Return only the finished Markdown document. Do not include commentary before or after it. Do not wrap the entire document in a code fence.